

Where Do Neural Networks Fail?

Post-Training Analysis and Novelty Detection Using Neural Network Self-Organizing Maps (NNSOM)

Capstone Group 5: Fyrooz Khan, Nazish Atta

DATS 6501 Capstone, Fall 2026, The George Washington University. Supervised by Dr. Amir Jafari.

Abstract

...

Keywords: self-organizing map, post-training analysis, error analysis, novelty detection, MNIST.

1. Introduction

A neural network can be right 99% of the time and still make mistakes that follow a pattern. Our starting model is a LeNet-5 network (LeCun et al. 1998). It reads handwritten digits from the MNIST dataset and gets 99.18% of the test images right (86 mistakes in 10,500 images). The question we ask is: where do these few mistakes come from?

Well-known tools such as saliency maps (Simonyan, Vedaldi, and Zisserman 2014), LIME (Ribeiro, Singh, and Guestrin 2016) and SHAP (Lundberg and S.-I. Lee 2017) explain one prediction at a time. They do not show where, inside the network, the mistakes gather.

We use a Self-Organizing Map (SOM) (Kohonen 1990) to get this wider view. A SOM places data on a flat grid so that similar items end up in nearby cells. We use the NNSOM library (Jafari n.d.). The plan is to train the CNN first, then record the 84 numbers it computes for each image just before it answers (the *embedding*), and then train a SOM on these embeddings. We ask three questions: (i) where on the map do the CNN's mistakes gather; (ii) can the map spot inputs that are unlike the training data; and (iii) can the map choose better samples for retraining than random choice? Because the SOM is trained on the CNN's own embeddings, we also check whether it tells us more than simple methods do, and we compare the plain CNN with a SOM-assisted CNN from start to finish (Figure 2).

Contributions. (1) A repeatable CNN and SOM pipeline for MNIST. (2) Evidence that the CNN's mistakes gather in a small set of cells of a 15×15 map, where digits overlap. This is tested against chance. (3) A comparison with simple baselines. It shows that this pattern is not special to the SOM, and that the CNN's own confidence is a much stronger signal for finding mistakes. (4) ...

2. Related Work

Our work touches four areas of research. For each one we say what it does and how our work is different.

CNNs as a source of features. LeNet-5 (LeCun et al. 1998) showed that convolutional networks learn useful features for handwritten digits. We do not propose a new classifier. We train LeNet-5 with the Adam optimizer (Kingma and Ba 2015) and then keep it fixed, so we can study how it organizes its features and where it fails.

Explaining single predictions. Saliency maps (Simonyan, Vedaldi, and Zisserman 2014) show which pixels change a class score the most. LIME (Ribeiro, Singh, and Guestrin 2016) fits a simple

model around one prediction. SHAP (Lundberg and S.-I. Lee 2017) gives each input feature a share of the credit. All three answer the question "why this prediction?". We ask a different question, "where in the network's features do mistakes gather?", so our work adds to theirs.

Self-Organizing Maps. Kohonen's SOM (Kohonen 1990) puts data on a flat grid so that nearby cells hold similar data. Vesanto and Alhoniemi (Vesanto and Alhoniemi 2000) showed how to group the cells of a trained SOM and measure the result. NNSOM (Jafari n.d.) provides the training code and the quality measures we use. SOM-VAE (Fortuin et al. 2019) and DESOM (Forest et al. 2021) train the features and the map together. We do the opposite on purpose. We keep the CNN fixed, so any signal from the SOM can be judged on its own.

Finding systematic errors. Domino (Eyuboglu et al. 2022) looks for groups of data where a model does badly and describes them in words. The Spotlight (d'Eon et al. 2022) searches the feature space for a region where the model's loss is high. These are closest to our error analysis. We differ in two ways. We use a map that keeps neighbors close, which gives one fixed picture that a person can read. And we test every claim against chance and against simple baselines.

Novelty and out-of-distribution detection. Hendrycks and Gimpel (Hendrycks and Gimpel 2017) proposed the maximum softmax probability (the network's top probability) as a simple baseline for finding mistakes and unfamiliar inputs. ODIN (Liang, Li, and Srikant 2018) improves it with a temperature setting and small changes to the input. Lee et al. (K. Lee et al. 2018) measure distance to each class in feature space, and Liu et al. (Liu et al. 2020) use an energy score. Deep SVDD (Ruff et al. 2018) learns a compact description of normal data, and Sun et al. (Sun et al. 2022) show that the distance to the nearest training features works well. The survey by Yang et al. (Yang et al. 2024) puts these methods side by side. We do not claim that a SOM beats them. We test whether the SOM's own measures give a useful novelty signal, while also giving a picture of where mistakes sit.

How our work differs. Earlier work either explains single predictions, trains maps together with the model, or detects novelty without showing where the failures sit. We use one SOM on fixed features for all three jobs: finding mistakes, detecting novelty, and guiding retraining. We test each claim against chance and against simple baselines. This already changed one conclusion: on MNIST the pattern of mistakes is real, but a plain k-means grouping and the CNN's own confidence do as well or better at finding them (Table 2).

3. Method

3.1 Data and model

MNIST is split once into 49,000 training, 10,500 validation and 10,500 test images. The split keeps the same share of each digit in every part, and it uses a fixed random seed so it can be repeated. We train a LeNet-5 with Adam (Kingma and Ba 2015) and keep the version with the best validation accuracy. For every image we then record the 84 numbers from the network's second-to-last layer (called fc2). These numbers are the embedding. We rescale them to the range -1 to 1, using the training data only (Figure 1).

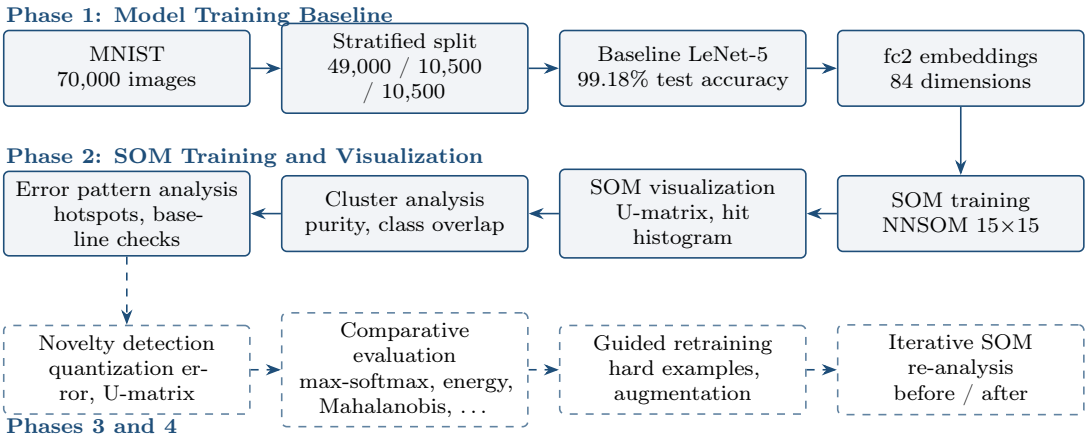


Figure 1. *The project pipeline. Solid boxes: Phases 1 and 2. Dashed boxes: Phases 3 and 4.*

3.2 Self-Organizing Map

We train a 15×15 map (225 cells) on the 49,000 training embeddings. Each image is assigned to the cell whose weights are closest to it, called its best-matching neuron (BMU). We judge the map with two numbers. The quantization error (QE) is the average distance between an image and its cell. The topological error (TE1) is the share of images whose two closest cells are not neighbors on the grid.

We tried three grid sizes (10×10, 15×15 and 20×20) under the same conditions: the same data, three starting neighborhood sizes, ten maps per grid, and validation data only. Larger grids fit the data more closely (QE 1.21, 1.06 and 0.98) and have slightly purer cells (97.6%, 98.2% and 98.4%), but they also have a higher topological error (TE1 4.8%, 6.6% and 7.7%). The grids are not clearly different at predicting the CNN's mistakes: the AUROC difference between 20×20 and 15×15 is +0.001 (95% range −0.100 to +0.089), so the comparison is inconclusive. Since no grid is clearly better, we use the 15×15 map for the main analysis. It has good purity and a lower topological error than the 20×20 map. We repeat the error analysis on the other two grids to check that the conclusions do not depend on this choice.

3.3 Finding where mistakes gather

For each cell we compute its *purity* (the share of its images that belong to its most common digit) and the *entropy* of its digit mix (how mixed the digits are). We compare the overall purity with a chance level. To get the chance level, we shuffle the digit labels 1,000 times and measure the purity again.

To find hotspots, we rank the cells by the lower end of the 95% Wilson interval (Wilson 1927) of their error rate. This keeps small cells from ranking high by luck. The top max(5, 5% of the cells) are the hotspots. We pick hotspots using a random half of the validation images and measure them on the other half, using the *concentration ratio*:

$$\text{ratio} = \frac{\text{share of the mistakes that fall in hotspots}}{\text{share of the images that fall in hotspots}}.$$

A ratio of 1 means the mistakes are spread evenly. We repeat this over 200 random half-splits and report the median. Chance is estimated by repeating everything 200 times with the mistake labels shuffled. We also compute the Spearman correlation between a cell's error rate and its purity and entropy, to see whether mistakes sit where digits overlap.

3.4 Comparison with simple baselines

We repeat the analysis with three simple methods: (a) k-means with 225 clusters (the same number as SOM cells); (b) the impurity of the 15 nearest training embeddings; and (c) the network's maximum softmax probability (Hendrycks and Gimpel 2017), meaning how sure the network is of its answer. Each score is judged on the same validation images by the area under the ROC curve (AUROC), which says how well the score separates mistakes from correct answers (1 is perfect, 0.5 is chance). We give bootstrap confidence intervals. None of these methods depends on the dataset, so they can also be used on Fashion-MNIST and CIFAR-10.

3.5 Phase 3: Novelty detection

...

3.6 Phase 4: Guided retraining

...

3.7 End-to-end comparison

Two paths are compared on the same test images (Figure 2). In path 1 (CF1), an image goes through the CNN and gets a prediction. In path 2 (CF2), the image first goes through the SOM/graph stage and then through the CNN. Each path gives a confusion matrix, CF1 and CF2. CF1 is the confusion matrix of the baseline CNN on the test images, the same one as in Phase 1 (Figure 4). The comparison shows how the SOM stage changes the predictions. The SOM/graph stage works as follows: ...

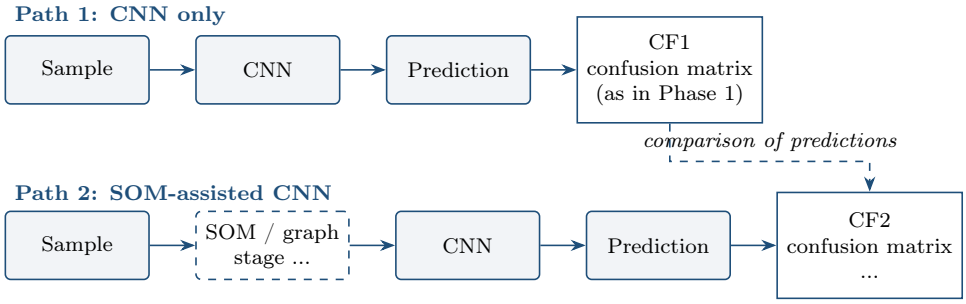


Figure 2. *End-to-end comparison: sample → CNN → prediction (CF1) against sample → SOM/graph → CNN → prediction (CF2), on the same samples. Dashed parts: ...*

4. Results

4.1 Leakage control

All development choices use the training and validation images only. The scaler, the SOM, k-means and the nearest-neighbor reference set are fitted on training embeddings. All error statistics are computed on validation images. The test images were used once, to score the baseline CNN, and the SOM analysis does not use them. The evaluation rules (grid comparison, hotspot rule, number of splits and shuffles, seeds) were set before the results were looked at. One caution: we chose the CNN checkpoint using the validation images, so these images are not completely untouched by model development.

4.2 Phase 1: Baseline CNN

The LeNet-5 gets 99.18% of the test images right (10,414 of 10,500; 95% Wilson interval 98.99–99.34%), with a macro F1-score of 0.9918. It makes 92 mistakes on the validation images (its average confidence is 0.997 when it is right and 0.833 when it is wrong) and 29 on the 49,000 training images. The most common validation confusions are $5 \to 0$, $9 \to 4$ and $4 \to 9$ (six each).

4.3 Phase 2: SOM structure and where mistakes gather

On the validation images the 15×15 map has a QE of 1.041 (training: 1.039) and a TE1 of 6.18%. Its cells follow the digit classes: the purity is 98.19% on validation images, against 17.4% (95% range 17.0–17.7%) with shuffled labels ($p = 0.001$).

The CNN's mistakes gather in a few hotspot cells (Figure 3 and Table 1). The 11 hotspot cells of the 15×15 map hold 11.1 times their fair share of the mistakes (95% range 6.1–17.6), against about 1 by chance ($p = 0.005$). The result is similar on the 10×10 and 20×20 maps. The error rate of a cell goes down as its purity goes up, and goes up as its digit mix grows, far beyond chance on all three grids. A finer grid does not predict mistakes better (the AUROC difference between 20×20 and 15×15 is +0.001, range −0.100 to +0.089), but with only 92 mistakes we cannot detect differences smaller than about 0.1 AUROC.

Measure (validation images)	**10×10**	**15×15**	**20×20**
Hotspot cells	5	11	20
Concentration ratio (95% range)	10.5 (5.4–17.3)	11.1 (6.1–17.6)	13.4 (8.0–19.3)
Chance level (median)	0.98	0.94	0.93
Spearman, error rate vs. purity	−0.755	−0.810	−0.766
Spearman, error rate vs. digit mix (entropy)	+0.778	+0.816	+0.768

Table 1. *Where mistakes gather, on three grid sizes (92 validation mistakes). All $p = 0.005$. The ratios should not be used to compare grids with each other.*

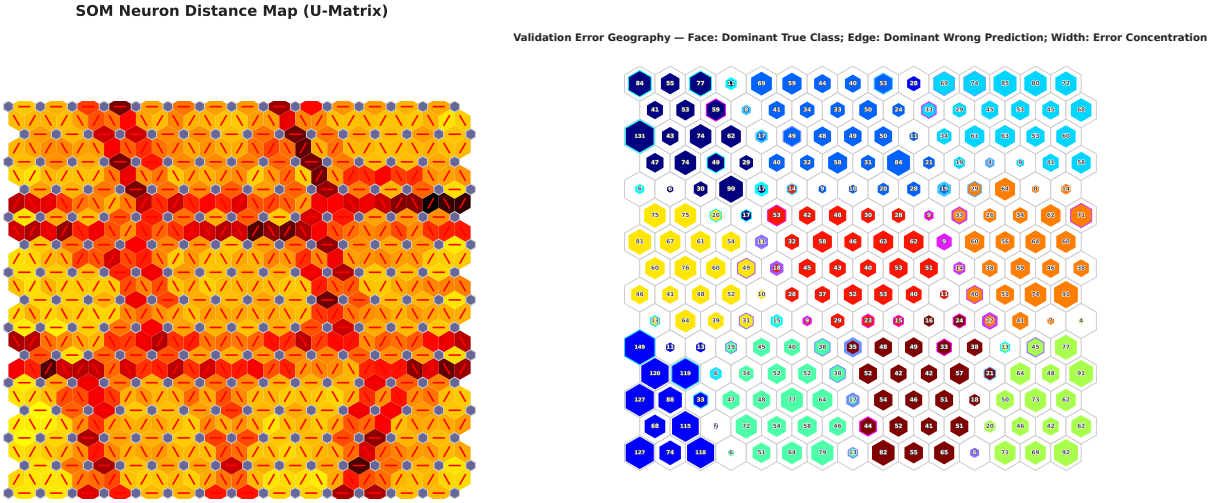


Figure 3. *Left: U-matrix of the 15×15 SOM (high values mark borders between groups of cells). Right: validation error map (fill: most common true digit; edge: most common wrong answer; size: how many mistakes).*

4.4 Comparison with baselines

Table 2 compares the SOM with the baselines on the same validation images. A SOM cell and a k-means cell find mistakes equally well (AUROC 0.685 and 0.693, with overlapping ranges).

Nearest-neighbor impurity does much better (0.874), and the CNN's own confidence does best (0.981). The SOM's hotspots are smaller than those of k-means (1.6% of the images against 3.8%), and they catch a similar share of the mistakes (18% against 20%). This is why the SOM has a higher concentration ratio. The hotspot-size numbers are exploratory.

Score	**AUROC (95% range)**	**Avg. precision**	**Hotspot ratio**	**Images in hotspots**	**Errors in hotspots**
SOM 15×15 cell error rate	0.685 (0.624–0.738)	0.053	11.1 (6.1–17.6)	1.6%	18.1%
k-means, 225 clusters, cell error rate	0.693 (0.651–0.729)	0.034	5.6 (4.6–6.1)	3.8%	20.4%
kNN impurity ($k = 15$)	0.874 (0.829–0.917)	0.228	–	–	–
CNN max-softmax probability	0.981 (0.974–0.987)	0.336	–	–	–

Table 2. *How well each score finds the 92 validation images that the CNN gets wrong (10,500 images in total). AUROC ranges come from repeated half-splits for the cell scores, and from bootstrap resampling for kNN impurity and max-softmax. Hotspot ratio: share of mistakes divided by share of images in the 11 hotspot cells, chosen on one half and measured on the other. For k-means: median of 10 seeds, with the lowest and highest in brackets. Chance level is about 1 ($p = 0.005$ for the SOM and for every k-means seed).*

4.5 Phase 3: Novelty detection

Novelty score	**AUROC**	**AUPR**	**FPR at 95% TPR**
SOM quantization error	...	...	...
SOM U-matrix distance	...	...	...
Maximum softmax probability	...	...	...
Energy / Mahalanobis	...	...	...
Isolation Forest / One-Class SVM / autoencoder	...	...	...

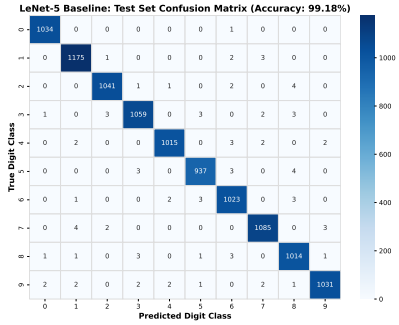
Table 3. *Novelty detection with MNIST as normal data: ...*

4.6 Phase 4: Guided retraining

Selection	**Test accuracy**	**Hotspot ratio**	**Map purity**
No retraining (baseline)	99.18%	11.1	98.19%
Random selection	...	...	...
SOM-guided selection	...	...	...

Table 4. *Guided retraining against random selection: ...*

4.7 End-to-end comparison



CF1: sample → CNN → prediction

CF2: sample → SOM/graph → CNN → prediction

...

Figure 4. *Confusion matrices of the two paths on the same test images. Left (CF1): the baseline CNN, the same matrix as in Phase 1. Right (CF2): to be filled after the SOM/graph stage has been run.*

5. Discussion and Limitations

On MNIST the SOM turns the 84 numbers of each embedding into a map where the digit classes form clean regions and the mistakes sit in the few cells where digits mix. This holds on three grid sizes and against chance. But if the only goal were to find likely mistakes, the maximum softmax probability would be the better tool. So the value of the SOM lies in the picture it gives, and possibly in novelty detection and retraining. Mistakes are also farther from their cell than correct images are (average distance 1.349 against 0.960), which may help find unfamiliar inputs. ...

Limitations. (1) We have only 92 validation mistakes, so the intervals are wide, and differences below about 0.1 AUROC cannot be seen. (2) MNIST is easy. Harder datasets (Fashion-MNIST, CIFAR-10) are needed before we can draw general conclusions. (3) The grid comparison could not separate the three grid sizes, so the choice of 15×15 rests on a trade-off between purity and topological error, not on a clear winner. (4) The baseline comparison covers only the 15×15 map and uses three k-means starts per seed. (5) We also used the validation images to choose the CNN checkpoint.

6. Conclusion

...

References

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE* 86.11, pp. 2278–2324.

Simonyan, K., Vedaldi, A., and Zisserman, A. (2014). "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps". *International Conference on Learning Representations (Workshop Track)*. URL: https://arxiv.org/abs/1312.6034.

Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). ""Why Should I Trust You?": Explaining the Predictions of Any Classifier". *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144.

Lundberg, S. M. and Lee, S.-I. (2017). "A Unified Approach to Interpreting Model Predictions". *Advances in Neural Information Processing Systems*. Vol. 30.

Kohonen, T. (1990). The Self-Organizing Map. *Proceedings of the IEEE* 78.9, pp. 1464–1480.

Jafari, A. (n.d.). NNSOM: Neural Network Self-Organizing Maps in Python. The George Washington University. URL: https://amir-jafari.github.io/SOM/.

Kingma, D. P. and Ba, J. (2015). "Adam: A Method for Stochastic Optimization". *International Conference on Learning Representations*. URL: https://arxiv.org/abs/1412.6980.

Vesanto, J. and Alhoniemi, E. (2000). Clustering of the Self-Organizing Map. *IEEE Transactions on Neural Networks* 11.3, pp. 586–600.

Fortuin, V., Hüser, M., Locatello, F., Strathmann, H., and Rätsch, G. (2019). "SOM-VAE: Interpretable Discrete Representation Learning on Time Series". *International Conference on Learning Representations*. URL: https://openreview.net/forum?id=rygjcsR9Y7.

Forest, F., Lebbah, M., Azzag, H., and Lacaille, J. (2021). Deep Embedded Self-Organizing Maps for Joint Representation Learning and Topology-Preserving Clustering. *Neural Computing and Applications* 33, pp. 17439–17469.

Eyuboglu, S., Varma, M., Saab, K., Delbrouck, J.-B., Lee-Messer, C., Dunnmon, J., Zou, J., and Ré, C. (2022). "Domino: Discovering Systematic Errors with Cross-Modal Embeddings". *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2203.14960.

d'Eon, G., d'Eon, J., Wright, J. R., and Leyton-Brown, K. (2022). "The Spotlight: A General Method for Discovering Systematic Errors in Deep Learning Models". *ACM Conference on Fairness, Accountability, and Transparency*. URL: https://arxiv.org/abs/2107.00758.

Hendrycks, D. and Gimpel, K. (2017). "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks". *International Conference on Learning Representations*. URL: https://openreview.net/forum?id=Hkg4TI9xl.

Liang, S., Li, Y., and Srikant, R. (2018). "Enhancing the Reliability of Out-of-Distribution Image Detection in Neural Networks". *International Conference on Learning Representations*. URL: https://openreview.net/forum?id=H1VGkIxRZ.

Lee, K., Lee, K., Lee, H., and Shin, J. (2018). "A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks". *Advances in Neural Information Processing Systems*. Vol. 31.

Liu, W., Wang, X., Owens, J. D., and Li, Y. (2020). "Energy-Based Out-of-Distribution Detection". *Advances in Neural Information Processing Systems*. Vol. 33.

Ruff, L., Vandermeulen, R., Goernitz, N., Deecke, L., Siddiqui, S. A., Binder, A., Müller, E., and Kloft, M. (2018). "Deep One-Class Classification". *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. PMLR, pp. 4393–4402.

Sun, Y., Ming, Y., Zhu, X., and Li, Y. (2022). "Out-of-Distribution Detection with Deep Nearest Neighbors". *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. PMLR, pp. 20827–20840.

Yang, J., Zhou, K., Li, Y., and Liu, Z. (2024). Generalized Out-of-Distribution Detection: A Survey. *International Journal of Computer Vision* 132.12, pp. 5635–5662.

Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association* 22.158, pp. 209–212.